

BirdCLEF Competition: Automatic Classification of Bird Calls through Deep Learning

Mauro Carlucci - m.carlucci66@studenti.uniba.it

University of Bari Aldo Moro, Bari, Italy

ABSTRACT

This work presents a comprehensive description of the workflow developed for participation in the Kaggle BirdCLEF 2025 competition. The project details the technologies and strategies implemented, including the design of a convolutional neural network-based pipeline for audio data analysis and preprocessing through mel-scale spectrograms. The approach was informed by an extensive review of scientific literature and analysis of state-of-the-art techniques proposed in previous competitions, with the aim of achieving the best possible ranking. This paper systematically documents all significant experiments, methodological choices, and their respective contributions to the final solution, highlighting the iterative process of optimization and the impact of each component on the overall performance. The results demonstrate the effectiveness of the adopted deep learning techniques for bioacoustic signal processing and species identification, culminating in an F1-score of 0.796 in classifying 206 animal species.

1. INTRODUCTION

Automatic recognition of bird calls represents an important challenge in the field of bioacoustics and environmental conservation. The BirdCLEF competition provides a dataset of labeled audio recordings for the development of automatic classification systems, enabling researchers to advance the state-of-the-art in acoustic species identification.

The objective of this project is to develop a workflow capable of automatically identifying animal species from their vocalizations, contributing to biodiversity monitoring efforts. Such systems have significant applications in ecological research, conservation biology, and automated wildlife monitoring systems.

2. DATASET AND PREPROCESSING

2.1. Dataset Description

The BirdCLEF dataset contains labeled audio recordings representing 206 different animal species with varying duration. Audio clips exhibit diverse quality levels, rated from 0 to 5, and generally include environmental noise and background sounds from other species beyond the primary labeled species.

Training data consisted of labeled recordings stored in the “train_audio” folder, with additional unlabeled data provided for optional use. The competition evaluation process involved submitting code that would be executed on a hidden test set of 700 one-minute soundscape files (“test_soundscape” folder)

containing natural audio recordings.

For a valid submission, participants were required to generate predictions for each 5-second segment of the test soundscapes, providing probability estimates for all 206 species in each temporal window.

The dataset includes diverse acoustic environments and recording conditions, providing a realistic challenge for automatic classification systems in natural soundscape analysis.

2.2. Audio Preprocessing

Audio recordings were processed through a comprehensive preprocessing pipeline designed to standardize input format while preserving acoustic characteristics relevant for species identification. The preprocessing workflow consists of four main stages: segment extraction, spectral transformation, normalization, and data augmentation.

2.2.1. Segment Extraction and Duration Optimization

Given the variable duration of audio recordings in the dataset (ranging from seconds to minutes), a systematic approach was developed to extract fixed-duration segments. Multiple configurations were evaluated to optimize both segment duration and extraction strategy.

Duration Analysis: Three different segment durations were tested (5, 10, and 15 seconds), with **5-second segments** providing the optimal balance between computational efficiency and acoustic information capture.

Extraction Strategies: Three different segment selection approaches were evaluated: center extraction (single segment from the middle of the recording), fixed multi-segment extraction (three segments from beginning, center, and end), and random single segment extraction.

After systematic evaluation, the optimal configuration was determined to be **5-second segments with random extraction**, providing the best trade-off between performance and computational efficiency while introducing beneficial variance during training.

2.2.2. Spectral Transformation

To extract meaningful features from audio signals for classification, we converted each waveform into a mel spectrogram—a time-frequency representation that emphasizes perceptually relevant information.

What are Mel Spectrograms?

Mel spectrograms represent audio in terms of time and frequency, using a frequency scale (mel scale) that mimics human pitch perception. Unlike linear scales, the mel scale spaces frequency bins more densely at lower frequencies, aligning better with how humans (and many animals) perceive sound.

The transformation from frequency (Hz) to mel scale is given by:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (1)$$

where f is the frequency in Hz and m is the corresponding mel frequency.

Generation Process

The process of computing a mel spectrogram consists of:

1. **Short-Time Fourier Transform (STFT):** The signal is divided into overlapping frames, and each is transformed into the frequency domain using the Discrete Fourier Transform (DFT), implemented efficiently via FFT:

$$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j2\pi kn/N} \quad (2)$$

where $x(n)$ is the input frame of length N , and $X(k)$ is the complex-valued spectrum at frequency bin k .

2. **Power Spectrum Calculation:** The magnitude squared of the STFT yields the power spectral density estimate:

$$P(k) = |X(k)|^2 \quad (3)$$

3. **Mel Filter Bank Application:** A set of overlapping triangular filters, spaced according to the mel scale, is applied to the power spectrum:

$$S_{\text{mel}}(b) = \sum_{k=0}^{N/2} P(k) \cdot M_b(k) \quad (4)$$

where $M_b(k)$ is the response of the b -th mel filter. This maps the linear frequency spectrum into the perceptual mel scale.

Implementation Parameters

The following parameters were used to compute mel spectrograms from raw audio:

- **Sample rate:** 32,000 Hz – Determines time resolution and maximum frequency
- **Number of mel bands:** 224 – Number of filters applied, i.e., frequency bins in the mel scale
- **FFT size:** 2,048 samples – Frame size used in the STFT; defines frequency resolution (15.625 Hz)
- **Hop length:** 512 samples – Step size between frames; controls overlap and time resolution
- **Frequency range:** 48–16,000 Hz – Focused on the frequency range most relevant for bird vocalizations

The resulting mel spectrograms are converted to decibel scale through logarithmic compression and normalized using min-max scaling to ensure consistent input scaling for the neural network. Finally, spectrograms are resized to 224×224 pixels

using bicubic interpolation to match standard computer vision model inputs.

2.2.3. Normalization

The mel spectrograms undergo a power-to-decibel conversion followed by min-max normalization to ensure consistent input scaling across samples:

$$S_{\text{dB}} = 10 \log_{10} (\max(S_{\text{mel}}, 10^{-10})) \quad (5)$$

$$S_{\text{norm}} = \frac{S_{\text{dB}} - \min(S_{\text{dB}})}{\max(S_{\text{dB}}) - \min(S_{\text{dB}})} \quad (6)$$

This normalization step preserves the relative spectral dynamics while mapping all spectrogram values to the [0, 1] range, improving training stability and interpretability for species classification.

2.2.4. Data Augmentation

To address dataset imbalance and enhance model robustness, two distinct augmentation strategies were applied: one to artificially expand low-resource classes before training, and another directly during training to regularize the model.

Time-Domain Augmentations (used for class balancing)

- *Time Shifting:* Random temporal displacement within $\pm 15\%$ of clip duration, applied with rollover using the `Shift` transform.
- *Time Masking:* Random temporal segments are zeroed out using `TimeMask`, with mask durations between 1–3% of the total audio length.
- *Additive Gaussian Noise (tested only):* Synthetic noise with SNR between 20–40 dB added via `AddGaussianNoise`. Evaluated but excluded due to negative performance impact.

Frequency-Domain Augmentations (used for class balancing)

- *Band-Stop Filtering:* Random suppression of narrow frequency bands using `BandStopFilter`, with bandwidths between 5–30% and center frequencies ranging from 1 to 15 kHz.

Mixup for Data Expansion (used for class balancing) For classes with fewer than 40 training examples, a mixup strategy was applied to generate new interpolated samples:

$$x_{\text{mixed}} = \lambda x_i + (1 - \lambda) x_j \quad \text{with} \quad \lambda \sim \text{Beta}(\alpha, \alpha), \alpha = 0.4 \quad (7)$$

$$y_{\text{mixed}} = \lambda y_i + (1 - \lambda) y_j \quad (8)$$

To prevent clipping, amplitude normalization is applied post-mixup:

$$x_{\text{normalized}} = \frac{x_{\text{mixed}}}{\max(|x_{\text{mixed}}|) + \epsilon} \quad (9)$$

Spectrogram-Based Regularization (applied during training) In addition to class-balancing augmentation, two spectrogram-

based transformations were applied to all training examples (regardless of class size), with fixed probabilities:

- *XY Masking* ($p = 0.4$): Independent masking on the time (X) and frequency (Y) axes of the mel spectrogram. Each mask occludes 10–20% of the respective dimension, encouraging invariance to local dropouts.
- *Horizontal CutMix* ($p = 0.4$): A random time segment from one spectrogram is replaced by the corresponding region from another. This simulates abrupt vocal interruptions and promotes compositional learning.

Class-Conditional Augmentation Policy (for low-resource classes) Data balancing augmentations (mixup, time shift, band-stop, time masking) were selectively applied to classes with fewer than 40 training examples. The augmentation type was sampled from the following probability distribution:

- Mixup: 70%
- Time shifting: 15%
- Time shift + band-stop filter: 10%
- Time shift + time masking: 5%

Dataset Splitting and Validation Integrity The dataset was split into training and validation sets using a stratified strategy. To preserve evaluation integrity:

- The validation set did not contain any class with fewer than or equal to 2 examples.
- Class-balancing augmentations were performed exclusively on the training set to prevent augmented samples from contaminating validation metrics.
- No test set was constructed locally, as the official test set was hidden during the competition and used for final evaluation.

Additionally, strategic downsampling was applied to highly represented classes (≥ 180 examples) by removing 50% of recordings with quality ratings ≤ 3 , reducing dataset imbalance while preserving high-quality training data.

3. METHODOLOGY

3.1. Model Architecture

The classification system is built upon the EfficientNet family of convolutional neural networks, specifically leveraging transfer learning from pretrained models to address the bird call classification task.

EfficientNet Architecture Overview EfficientNet represents a systematic approach to scaling convolutional neural networks that achieves superior performance through compound scaling of network depth, width, and resolution. Unlike traditional scaling methods that arbitrarily increase one dimension, EfficientNet uses a compound coefficient to uniformly scale all three dimensions according to a set of fixed scaling coefficients, optimizing the trade-off between accuracy and computational efficiency.

EfficientNet-B0 Base Model

The EfficientNet-B0 architecture begins with a stem layer that prepares the input for deeper processing. This initial stage consists of:

- **Input layer:** A $224 \times 224 \times 3$ image, where mel spectrograms are replicated across three channels
- **Convolution:** A 3×3 convolution with 32 filters and stride 2 to reduce spatial resolution
- **Batch Normalization:** Helps stabilize and accelerate training

This processed output is then passed to the main part of the network, which includes a series of MBConv blocks and a custom classification head tailored for the task. MBConv (Mobile Inverted Bottleneck Convolution) blocks combine depthwise separable convolutions with squeeze-and-excitation and residual connections, allowing for efficient feature extraction with reduced computational cost.

Evaluated Model Variants Three specific EfficientNet implementations were systematically evaluated:

1. **EfficientNet-B0 (ImageNet pretrained):** Standard EfficientNet-B0 pretrained on the ImageNet-1K dataset, providing general visual feature representations transferable to spectrogram analysis.
2. **tf_efficientnet.b0.ns_jft.in1k:** An enhanced version incorporating Noisy Student training methodology. The Noisy Student (NS) technique involves iterative semi-supervised learning where a teacher model generates pseudo-labels for unlabeled data, and a student model learns from both labeled and pseudo-labeled data with added noise during training. This model was pretrained on the larger JFT dataset before fine-tuning on ImageNet, resulting in improved robustness and generalization capabilities.
3. **EfficientNetV2:** EfficientNetV2 is a more advanced and complex evolution of the original EfficientNet-B0. While both models are based on compound scaling, EfficientNetV2 introduces deeper and wider architectures with significantly more layers and parameters, allowing it to capture richer and more detailed features.

Compared to EfficientNet-B0, it includes:

- **More convolutional layers:** Increasing network depth improves its ability to model complex patterns in data.
- **Fused MBConv blocks:** A modified version of MBConv that simplifies early layers and improves training speed, while adding structural complexity.
- **Higher capacity:** More channels and larger hidden dimensions make the model more expressive, at the cost of greater computational load.

This added complexity makes EfficientNetV2 more powerful, but also more resource-intensive, making it suitable for tasks where higher accuracy justifies the increased model size.

3.2. Classification Head

Two distinct classification head architectures were implemented and evaluated to optimize performance for bird call classification: a traditional Multi-Layer Perceptron (MLP) approach and a Sound Event Detection (SED) framework.

Multi-Layer Perceptron Classification Head The MLP approach employs a straightforward deep classification architecture built on top of the EfficientNet backbone:

- **Input layer:** Receives 1280-dimensional feature vectors from EfficientNet-B0 global average pooling
- **Dropout layer:** 40% dropout rate for initial regularization
- **Hidden layer:** Linear transformation to 512 neurons with ReLU activation
- **Batch normalization:** Applied to 512-dimensional features for training stability
- **Secondary dropout:** 30% dropout rate for additional regularization
- **Output layer:** Linear transformation to 206 neurons (one per species) with sigmoid activation for multi-label classification

This architecture provides a direct mapping from global audio features to species probabilities, offering computational efficiency and interpretable outputs suitable for clip-level classification tasks.

Sound Event Detection (SED) Classification Head The SED approach implements a sophisticated temporal attention mechanism designed to capture both clip-level and frame-level audio patterns:

Architecture Components:

- **Temporal feature processing:** EfficientNet features averaged across frequency dimension, preserving temporal structure
- **Channel smoothing:** Combination of max-pooling and average-pooling (kernel size 3) for noise reduction

Attention Block (AttBlockV2): The core innovation lies in the dual-branch attention mechanism:

- **Attention branch:** Generates normalized attention weights via $\text{softmax}(\tanh(\text{Conv1D}(x)))$
- **Classification branch:** Produces frame-level predictions through Conv1D with sigmoid activation
- **Weighted aggregation:** Combines outputs as $\sum(\text{attention} \times \text{classification})$

Multi-scale Output Generation:

- **Clipwise output:** Attention-weighted aggregation for single clip prediction
- **Framewise output:** Temporal interpolation (30:1 ratio) providing fine-grained localization
- **Segmentwise output:** Intermediate temporal resolution for segment-level analysis

Theoretical Advantages: Unlike the MLP approach, SED maintains temporal structure throughout processing, enabling:

- **Temporal localization:** Precise identification of when bird calls occur within clips
- **Attention-based aggregation:** Intelligent weighting of temporal segments based on acoustic relevance
- **Multi-resolution analysis:** Simultaneous processing at multiple temporal scales
- **Robust handling of complex soundscapes:** Distinction between target vocalizations and background noise

This temporal awareness is crucial for the BirdCLEF evaluation format, which requires predictions for 5-second segments within 60-second soundscapes. The SED architecture naturally handles the competition’s segmentation requirements while providing interpretable attention patterns that indicate when and where bird calls occur.

3.3. Training Configuration

Several combinations of training hyperparameters were tested to identify the most effective setup. The configuration that yielded the best results is reported below:

- **Optimizer:** AdamW, with a base learning rate of 4×10^{-4} for the backbone and 8×10^{-4} for the classification head
- **Learning rate scheduler:** CosineAnnealingLR, $T_{max} = 30$ epochs, $\eta_{min} = 1 \times 10^{-6}$
- **Batch size:** 96
- **Loss function:** Binary Cross-Entropy with Logits (BCE-WithLogitsLoss), suitable for multi-label classification
- **Training epochs:** 30, with early stopping (patience = 5 epochs without improvement)
- **Train/validation split:** 80%/20%, stratified by class, excluding classes with fewer than 3 samples from the validation set

This configuration was selected after extensive experimentation, balancing convergence speed, generalization, and computational efficiency.

4. EXPERIMENTAL RESULTS

4.1. Evaluation Metrics

Model performance was evaluated using a two-step approach, reflecting both the competition requirements and standard machine learning practices.

1. Local Validation: During model development, performance was assessed on a held-out validation set, created via stratified splitting of the training data (excluding classes with fewer than 3 examples from validation). The following metrics were computed:

- Macro-averaged precision, recall, and F1-score
- Per-class performance analysis, with particular attention to rare and common classes
- Confusion matrix visualization

Metrics were calculated using thresholded sigmoid outputs (multi-label setting), and additional analyses were performed to compare results across class frequency categories (rare, normal, common).

2. Official Competition Evaluation: The final evaluation was performed on a hidden test set provided by the BirdCLEF competition organizers. For submission, the model generated probability predictions for each 5-second segment of the 60-second test soundscapes, covering all 206 species. The official leaderboard scores were based on macro-averaged F1-score computed on this secret test set, ensuring fair comparison between participants.

All reported results in this document refer to the best configuration found locally, while the final ranking and test performance were determined by the competition's automated evaluation system on the secret

4.2. Post-processing of Predictions for Submission

To maximize performance in the official Kaggle evaluation, advanced post-processing techniques were implemented on the model predictions, as documented in the inference notebooks. The main strategies adopted are:

- **Centered segmentation with strategic padding:** Each test soundscape is divided into 5-second segments, applying padding at the beginning and end of the audio signal to properly center the first and last segments, reducing the risk of information loss at the edges.
- **Test-Time Augmentation (Delta Shift TTA):** For each segment, multiple predictions are generated by applying small temporal shifts ($\pm 25\text{ms}$, $\pm 50\text{ms}$) to the audio signal. The resulting predictions are then averaged, increasing robustness to temporal variations and noise.
- **Advanced temporal smoothing:** Predictions for each segment are further filtered using a weighted moving average (kernel $[0.1, 0.2, 0.4, 0.2, 0.1]$) that combines predictions from adjacent segments, reducing local variability and improving the temporal consistency of the estimated probabilities.

These post-processing techniques, applied to both standard EfficientNet and SED models, enabled more stable and reliable predictions, improving the final

4.3. Results

The project achieved a steady improvement in performance through a series of experimental milestones, each introducing specific optimizations. The main steps and their impact on the macro F1-score (validation or public leaderboard) are summarized below:

- **v6-v7:** SimpleCNN, central segment, basic params
F1-score: 0.581
- **v10:** SimpleCNN, multi-segment (20%, 50%, 80%), class balancing (30%)
F1-score: 0.665
- **v11:** EfficientNet-B0, adaptive multi-segment dataset, batch size 96, different LR
F1-score: 0.712
- **v14:** EfficientNet, horizontal cutmix, reduced LR, CosineAnnealingLR, class balancing 40%
F1-score: 0.743
- **v20:** Optimized data augmentation, tuned training parameters
F1-score: 0.744
- **v23 (SED):** SED model, attention block, audio-specific augmentations, end-to-end
F1-score: 0.733
- **v24:** EfficientNet with optimized oversampling and random segments
F1-score: 0.765

- **v25:** EfficientNet-B0 NS JFT, XY masking ($p=0.4$), horizontal cutmix ($p=0.4$), aggressive balancing (50%, threshold=180)
F1-score: 0.789
- **v25 + Post-processing:** Advanced post-processing: temporal smoothing with kernel $[0.1, 0.2, 0.4, 0.2, 0.1]$, delta shift TTA ($\pm 25\text{ms}$, $\pm 50\text{ms}$), strategic padding
F1-score: 0.796

In summary, the final solution as implemented in `notebook-birdclef MLP.ipynb` and `inference MLP.ipynb` combines the best-performing configuration: fully randomized 5-second segments, EfficientNet-B0-NS-JFT, with a regularized MLP head, extensive class-aware data augmentation, and advanced post-processing (padding, delta shift TTA, temporal smoothing). This pipeline achieves a macro F1-score of **0.796** on the official BirdCLEF 2025 test set, confirming the effectiveness of the adopted strategies.

5. CONCLUSIONS

This study demonstrates the effectiveness of convolutional neural networks for automatic bird call classification in the BirdCLEF competition context. The mel-spectrogram representation combined with data augmentation techniques proved successful for capturing species-specific acoustic patterns.

Future work could explore ensemble methods to further boost classification performance, as well as more advanced pre-processing strategies aimed at removing human voices and other non-bird sounds from the dataset. Additionally, pseudo-labelling techniques could be applied to leverage the large amount of unlabeled audio data provided by the competition, which was not utilized in the current experiments. These directions may further improve the robustness and generalization of the developed system, which already shows promise for real-world applications in biodiversity

REFERENCES

- [1] BirdCLEF 2025 Competition. Kaggle Competition Page. <https://www.kaggle.com/competitions/birdclef-2025>
- [2] M. Tan and Q. V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *arXiv preprint arXiv:1905.11946*, 2020. <https://arxiv.org/abs/1905.11946>
- [3] A. Soni. Understanding the Mel Spectrogram. Medium, 2020. <https://medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53>
K. Kushwaha. Exploring Mel Spectrograms: A Powerful Feature Extraction Tool for Audio Signals. Medium, 2023. <https://medium.com/@kunalkushwahatg/exploring-mel-spectrograms-a-powerful-feature-extraction-tool-for-audio-signals-3b68ff6f>
- [4] K. Candrisolu. Transforming Audio to Mel Spectrogram - BirdCLEF 25. Kaggle Notebook. <https://www.kaggle.com/code/kadircandrisolu/transforming-audio-to-mel-spec-birdclef-25>
- [5] Multi layer perceptron Medium.com page. <https://medium.com/eincode/mastering-the-multi-layer-perceptron-mlp-for-image-classification-a0272baf1e29>
- [6] J. MacIsaac, S. Newson, A. Ashton-Butt, H. Pearce, B. Milner. Improving acoustic species identification using data augmentation within a deep learning framework. *Ecological Informatics*, 83, 2024, 102851. <https://doi.org/10.1016/j.ecoinf.2024.102851>
- [7] I. Jordal. Audiomentations: A Python library for audio data augmentation. <https://github.com/iver56/audiomentations>
- [8] H. Arai. Introduction to Sound Event Detection (SED). Kaggle Notebook. <https://www.kaggle.com/code/hidehisaarail213/introduction-to-sound-event-detection>
- [9] BirdCLEF 2025 - 1st Place Solution. Kaggle Competition Discussion. <https://www.kaggle.com/competitions/birdclef-2025/discussion/583577>
- [10] BirdCLEF 2024 - 1st Place Solution. Kaggle Competition Discussion. <https://www.kaggle.com/competitions/birdclef-2024/discussion/512197>
- [11] H. Arai. EfficientNetB0 with Sound Event Detection (SED) for BirdCLEF. Kaggle 6th Place Solution, 2021. <https://github.com/koukyo1994/kaggle-birdcall-6th-place/tree/master> <https://www.kaggle.com/competitions/birdsong-recognition/discussion/183204>
- [12] Bird25 OpenVino Ensemble Inference Baseline (LB 0.874). Kaggle Competition Notebook. <https://www.kaggle.com/code/kurisew/bird25-openvino-ensemble-infer-baseline-lb-874/notebook>
- [13] Kaggle Competition Discussion. <https://www.kaggle.com/competitions/birdclef-2025/discussion/583699>
- [14] BirdClef 2023 - Advanced Techniques and Methodologies. Kaggle Competition Discussion. <http://kaggle.com/competitions/birdclef-2023/discussion/412808>